UNITY BASIC 3D FIRST-PERSON PUZZLE GAME

Developing a 3D First-Person Puzzle project in Unity requires a few key crucial unity concepts that I'll be covering here in this document. The key here is to keep puzzle scripts responsible only for their own functionality, single responsibility principle (SRP), allowing clean code to scale nicely, without crowding other puzzles.

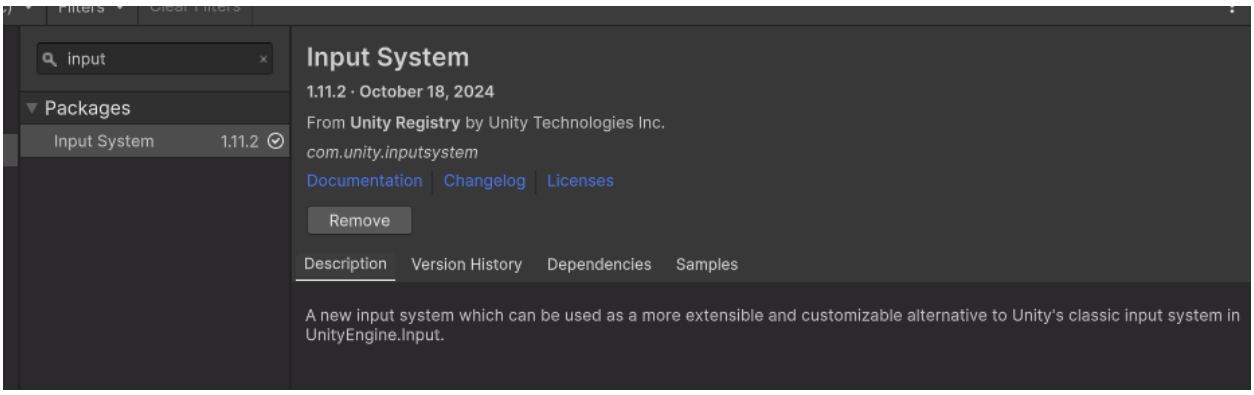
Start by opening a brand-new Unity 6 Universal Render Pipeline project, it's important to make sure that URP is used as materials may break when importing the package. After everything loads, use the package manager to import the base project.

IF AT ANY POINT SCRIPTS ARE NOT RECOMPILING ON THEIR OWN, PRESS CTRL+R WHILE IN THE UNITY EDITOR

Player Movement

First, we must set up the player input mappings:

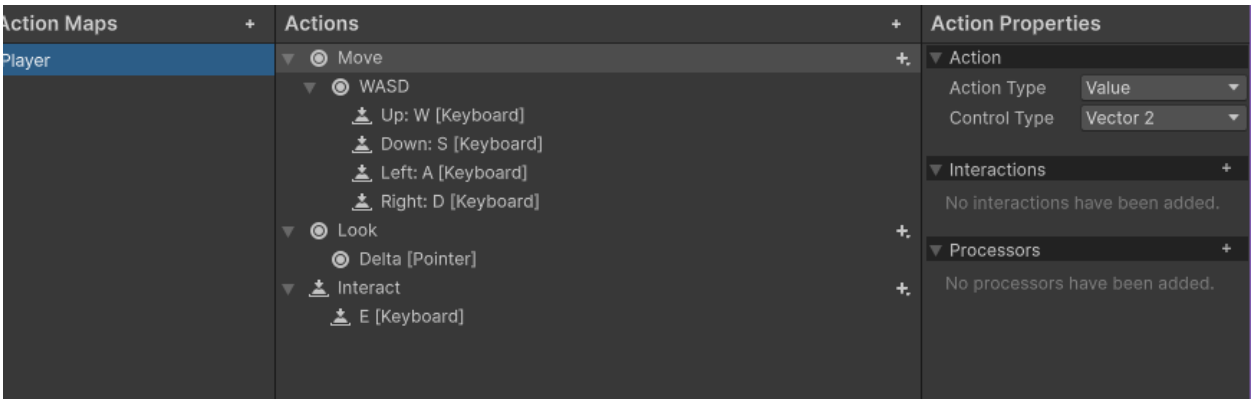
1. Open the Package Manager [Window > Package Manager]
2. Select Unity Registry and search for "Input" then import the Input System



3. Create a new Input actions asset by right clicking in the asset folder
4. Open the newly created Input System Actions
5. On the left-hand side, you will see "Action Maps", create a new map called "Player"
6. Add 2 new actions following this pattern:
    a. Name one of them "Move" and the other "Look"

    b. Set the Action Type to: Value
    c. Set the Control Type to: Vector2
7. For the Move action, right click on the action and add Up\Down\Left\Right Composite
    a. Delete the <No Binding> entry
    b. Set up the appropriate WASD bindings for each direction respectively
8. For the Look action, click on the <No Binding> and set the path to “delta [Pointer]”
9. Add a new action and name it “Interact” and set the <No Binding> path to the “E” key

The end result should look something like this:



10. Save this asset by hitting Ctrl+S and close the window
11. In the asset folder, left click on Input Systems Action asset you just modified
12. In the inspector, enable “Generate C# Class” and then name the Class Name to PlayerInput
13. Hit apply, a new C# script should have been generated wherever the class file was specified, it doesn’t matter where it is as long as it is within the project folder

And we are good to go! We can begin hooking up the input system to scripts for functionality.

Rigging the Input System for script functionality:

1. Create a new script and call it FPController
2. Let's set up the required variables needed to make the controller work.

```
public float moveSpeed = 7.5f;

private CharacterController controller;
private PlayerInput input;

private Vector2 inputVector;




private Vector3 velocity;
public float gravity = -9.81f;
public float groundCheckDistance = 0.4f;
public LayerMask groundMask;
private bool isGrounded;

public Transform groundCheck;
```

3. Create a new Awake() method and enter the following

```
private void Awake()
{
    input = new PlayerInput();
    controller = GetComponent<CharacterController>();
}
```

4. And now in a Start() method:

```
private void Start()
{
    input.Enable();

    // When the player moves, store the value in inputVector
    input.Player.Move.performed += read => inputVector = read.ReadValue<Vector2>();
    // When no longer moving, set inputVector to zero
    input.Player.Move.canceled += read => inputVector = Vector2.zero;
}
```

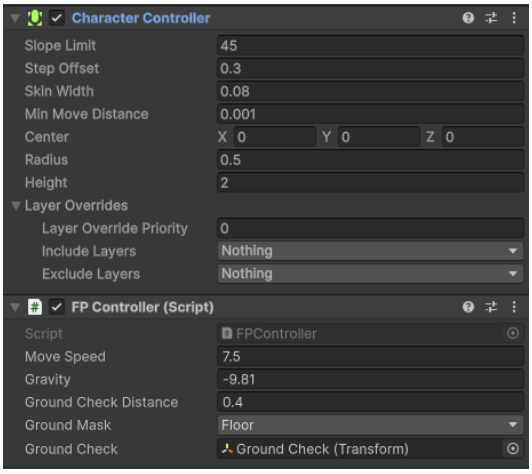
5. And lastly in our Update() method:

```
private void Update()
{
    Vector3 move = (transform.right * inputVector.x + transform.forward * inputVector.y).normalized;
    controller.Move(moveSpeed * Time.deltaTime * move);


    // Ground check
    isGrounded = Physics.CheckSphere(groundCheck.position, groundCheckDistance, groundMask);
    if (isGrounded && velocity.y < 0) velocity.y = -2f;

    // Gravity
    velocity.y += gravity * Time.deltaTime;
    controller.Move(velocity * Time.deltaTime);
}
```

6. Our FPController is ready to go. Find the player capsule located within the scene and attach the script as a component
7. Ensure also that the player capsule has a Character Controller component attached.
8. Assign the Ground Mask to Floor
9. There is a child game object under player named Ground Check, drag that game object into the FPController Ground Check variable



10. Now lets create another script and call it FPCamera
11. Assign the required variables:

```
public float mouseSensitivity;


[SerializeField] private Transform playerTransform;
private PlayerInput input;

private Vector2 lookInput;
private float xRotation = 0f;


public float interactDistance = 3f;
public LayerMask interactableLayer;
```

12. Repeat creating a new input in an awake method
    a. Simply writing the following will work: private void Awake() => input = new();
13. Start() will be little more complex… Note that Interact() will show an error until we complete steps 15-16. You can ignore the error it tries to give you for now

```
private void Start()
{
    input.Enable();

    input.Player.Look.performed += read => lookInput = read.ReadValue<Vector2>();
    input.Player.Look.canceled += read => lookInput = Vector2.zero;
    input.Player.Interact.performed += read => Interact();

    Cursor.lockState = CursorLockMode.Locked;
    Cursor.visible = false;
}
```

14. And now Update()

```
private void Update()
{
    float mouseX = lookInput.x * mouseSensitivity;
    float mouseY = lookInput.y * mouseSensitivity;

    xRotation -= mouseY;
    xRotation = Mathf.Clamp(xRotation, -90f, 90f);
    transform.localRotation = Quaternion.Euler(xRotation, 0f, 0f);

    playerTransform.Rotate(Vector3.up * mouseX);
}
```

15. To complete our player object, we need to add an interact method onto our FPCamera, but first we need to implement an interface that will allow us to easily call Interact() onto game objects that can be interacted with
    a. To do this, tab back into the Unity Editor and create another new script. Call it IInteractable
    b. Delete the entire base script and simply do this

```
public interface IInteractable
{
    void Interact();
}
```

    c. This will allow us to add this interface to our game object scripts that need to be interacted with
16. Jump back into the FPCamera script, we can now add this method

```
private void Interact()
{
    Ray ray = new(Camera.main.transform.position, Camera.main.transform.forward);

    if (Physics.Raycast(ray, out RaycastHit hit, interactDistance, interactableLayer))
    {
        var interactable = hit.collider.GetComponent<IInteractable>();
        interactable?.Interact();
    }
}
```

17. Close out of your code editor, and make sure that the new scripts recompiled

18. Add the new FPCamera script on top of the main camera nested under Player.

19. Adjust mouse sensitivity to your liking (mine had to be set to 0.1)

20. For Player Transform drag the player parent object into the field (it will automatically assign the transform)

21. Make sure the Interactable Layer is set to Everything

Puzzle Game Objects

Lets now get some functionality scripts so that we can start hooking up objects together to allow for

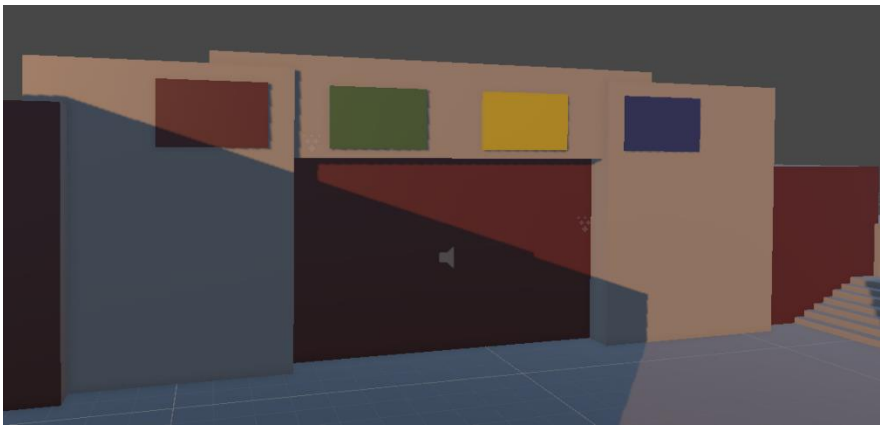
Lever:

1. Let's create a script for the lever object in the first area. Create a new script and call it Lever
2. This script should inherit from Monobehaviour and IInteractable
3. Add the following variables:
    a. public bool Activated
    b. public IInteractable affectedInteractable
4. Now we need to satisfy the interactable interface, add a public void Interact() method
    a. Now write: affectedInteractable.Interact ()
5. Go back into the editor and attach the lever script onto the lever game object.
6. Find the red door game object that connects area 1, to area 2. Open the Slide Down Door Script and attach IInteractable to this object as well so it inherits from it

It's as simple as that, all we need to do now is set a game object with an IInteractable behavior attached to get this to work. The script will automatically call "Interact" on the object to have it perform the desired action.

Area 2: Puzzle Color Sequence

Lets now add a little more logic to create more complex puzzles.



As you can see here, this puzzle implies that there is a specific sequence of colors that must be input. We can create a sort of “driver” script to manage each individual puzzle cleanly. We want to keep puzzles separated in their own logic, but reuse scripts for functionality.

Take a look around this area, you will see that there are multiple buttons scattered around here. Our job here is to rig up a script to function with our new interactable interface, and then process the data to simulate a color pattern matching sequence.

Color buttons:

1. First we need to create our own color definition, as Unity Colors by default are material colors, and thus not usable in our case.
2. Create a new script and call it Colors. If you find that potentially you might get confused by colors and color, you can name this script ColorSequence, this will just be a simple enum anyway.
3. Open up the script and once again, delete everything and simply write:

```
public enum Colors
{
    Red,
    Green,
    Blue,
    Yellow,
}
```

4. For now, this will suffice for what we need in this area, if you’d like to, you may add more colors if needed
5. Now we will need to set up our buttons and sequence driver to use our enum data
6. Create a new script and call it ColorOrderSequence, this will be the main logic container for this puzzle
7. Add these first

```
public List<Colors> correctSequence;

private IInteractable door;
private int currentIndex = 0;
```

8. Create an Awake() method and call GetComponent<IInteractable>() for the door
9. Now we will need 2 methods to properly process the order of colors the player inputs
10. Let's practice a little bit, I will describe the 2 methods required and describe what they are responsible for
    a. Method 1: CheckCurrentSequence
        i. Parameters: (Colors color)
        ii. Purpose: This function will check what color was just passed in and compare it to the correct color sequence at whatever index we are currently at. Meaning that if Red was passed in and the first element of correctSequence is Red, then we can successfully iterate the index, otherwise if it is wrong then we need to reset the current index.
    b. Method 2: HandleColorInput
        i. Parameters: (Colors color)
        ii. Purpose: This function will first check the current sequence using the first method we wrote CheckCurrentSequence. Then we need to check if the current index is greater than or equal to the correct sequence list count. If it is, then we need to call Interact() on the door interactable, and we can optionally reset the index back to 0.

Try it out! Once you take a shot at it scroll down to the next page to see how it compares to what I have.

```
public void HandleColorInput(Colors color)
{

    CheckCurrentSequence(color);

    if (currentIndex >= correctSequence.Count)
    {
        door.Interact();
        currentIndex = 0;
    }

}


private void CheckCurrentSequence(Colors color)
{
    if (color != correctSequence[currentIndex])
    {
        currentIndex = 0;
        audioSource.Play();
        return;
    }

    currentIndex++;
}
```

11. Save this file and tab back into the editor
12. Find the door that connects area 2 and area 3 (the door below the colored walls) and attach the new script we just made
13. Set the list to correct sequence shown above the door by adding new entries to the list [Red -> Green -> Yellow -> Blue]

Now let's make our button scripts and add the final glue to make everything work together.

14. Add a new script and call it ColorButton
15. For our case we are simply going to provide a reference of color order sequence to each individual button. For larger scale projects using practices like dependency injection or even singletons would be better, but for our case this is fine
16. Let's add the variables we need:

```
[SerializeField] private ColorOrderSequence colorOrderSequence;
[SerializeField] private Colors color;
```

17. This script is meant to be an entry point for Interact() meaning we need to add the IInteractable interface in the inheritance, go ahead and add it after MonoBehaviour
18. And since this is just a script that lives freely on its own, we can think of it like a runtime data container for whatever color the button represents. This means that all we need to do is add the Interact() method and call whatever function we need.

```
public void Interact()
{
    colorOrderSequence.HandleColorInput(color);
}
```

19. Now save this script, and close out of your code editor. In the unity editor we now need to add this script onto each button game object. It's important to note that the script should be added to the *PARENT* button game object (Do not attach this script to and Panel game object)

20. And then in the inspector, we need to assign each button a color order sequence reference, meaning that the door game object connecting area 2 and area 3 (where we attached the last script) and drag that into the button reference. Also set the Color to the appropriate color of the button
21. Do this for all 4 of the buttons found in the scene

And that's it! We have a simple scene set up with very basic puzzles that you can try out. Go ahead and give it a shot yourself to see if everything is rigged up correctly.

Your next task will be to extend upon this project to create your own puzzle. Use the same practices you just learned to add "actor" objects and rig them up to a driver script. There are a few important things to note while creating your own puzzles:

If you want to add more floors to your scene, ensure they have the Floor Layer assigned to it. Otherwise, the player controller may not act appropriately.

IF YOUR SCRIPTS ARE NOT RELOADING PRESS CTRL + R TO REIMPORT THEM

If you want to add walls, try to keep things organized in a single parent object for walls. Your scene hierarchy can become flooded with game objects and become very messy.